

Do Code Semantics Help? A Comprehensive Study on Execution Trace-Based Information for Code Large Language Models

Jian Wang, Xiaofei Xie, Qiang Hu*, Shangqing Liu, Yi Li

Motivation

Challenge: Code LLMs struggle with runtime behavior reasoning

- Limited proficiency in understanding program execution
- Inconsistent semantic information representation (execution traces)

Two Main Problems:

- Fine-tuning with semantic information - Does it help?

$$\mathcal{D} = \{ (x, y) \mid \bar{x} = (b, r), \bar{y} = a \}$$

(x, y) is the input-output pair for model fine-tuning, b denotes the buggy code fragment, r is execution-trace rationale, and a represents the patched code

- Inference with semantic information - Does test-time scaling improve?

$$y \sim \text{Target}(\theta, q, N, \text{Verify}) \quad (1)$$

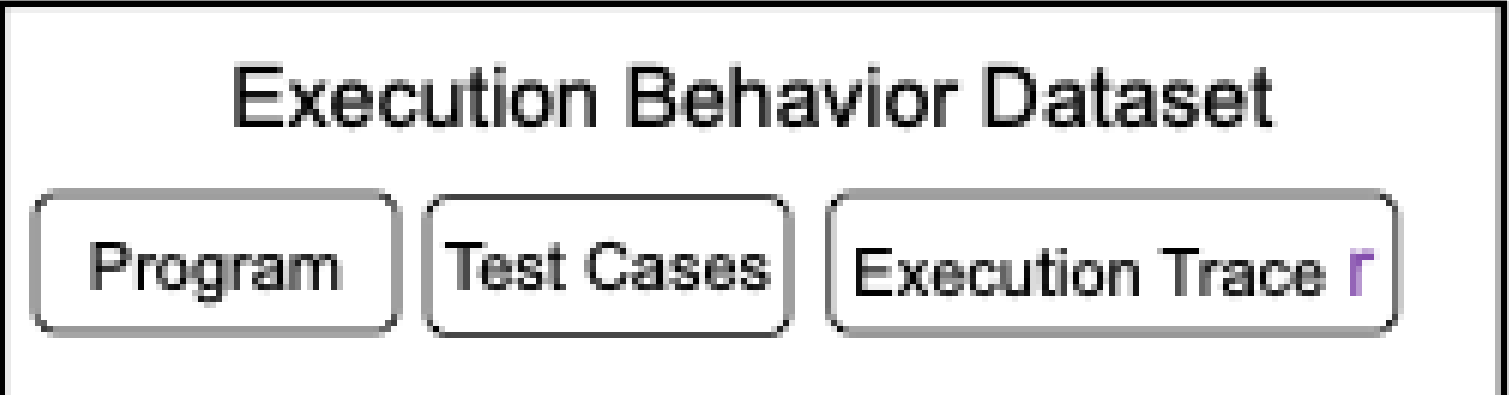
$$\theta_q^*(N) = \arg \max_{\theta} \left(\mathbb{E}_{y \sim \text{Target}(\theta, N, q, \text{Verify})} \left[\mathbb{1}_{y=y^*(q)} \right] \right) \quad (2)$$

The test-time compute-optimal strategy $\theta_q^*(N)$ maximizes the expected probability of generating the correct answer: where $\text{Target}(\theta, q, N, \text{Verify})$ which iterates through proposing, verifying, pruning, and refining partial solutions until the budget N is exhausted. $y^*(q)$ denotes the ground-truth, accuracy via the indicator $\mathbb{1}_{\{y=y^*(q)\}}$.

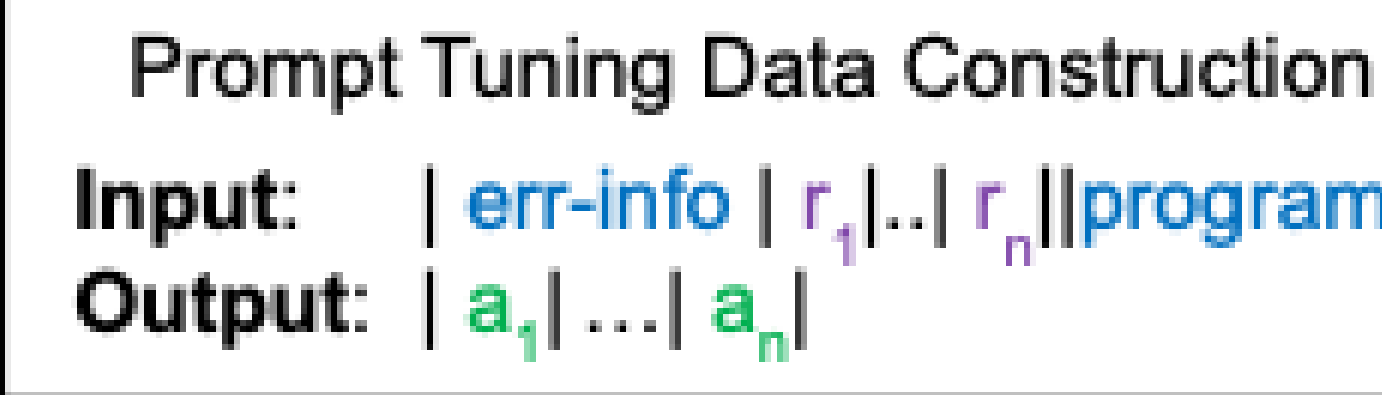
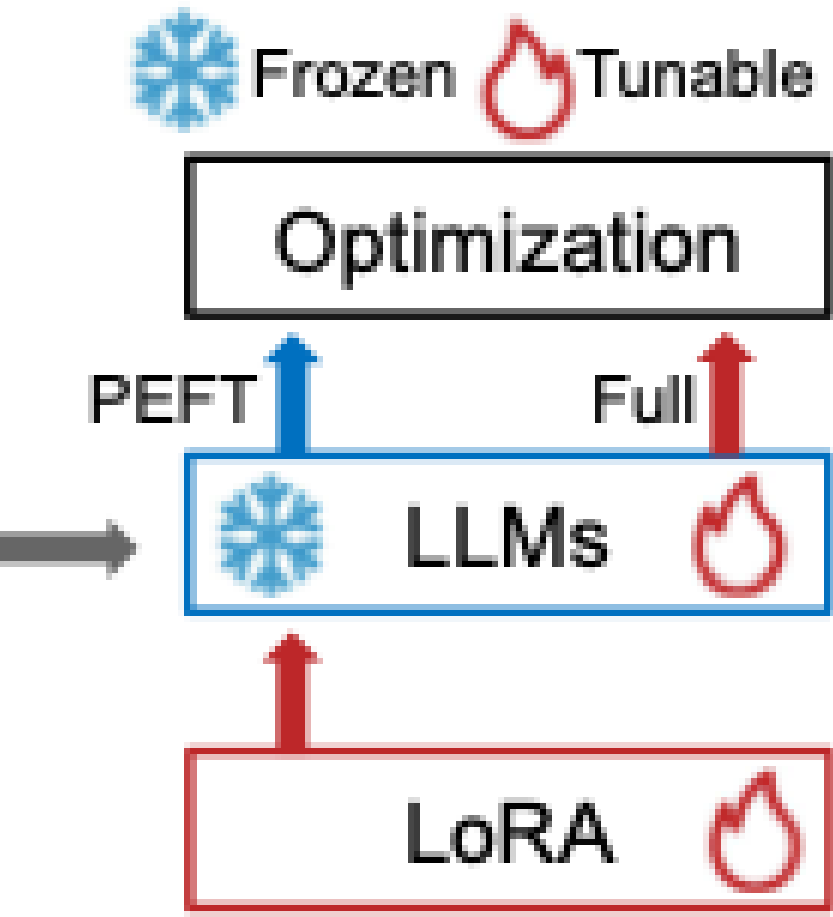
Framework Overview

Paradigm of our framework: runtime execution, formalizing, Finetune, Scaling Inference

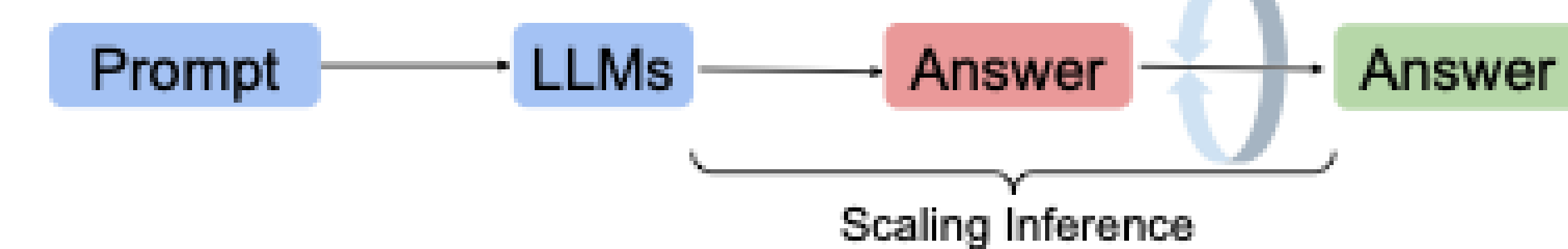
Dataset Curating



Finetune



Inference



Trace Representations

Different ways to represent execution semantics:

- NExT example with inline comments
- Concise format with separated state info
- SemCoder natural language description
- CodeExecutor detailed state tracking

Key difference:

Placement (inline vs separate) and verbosity (detailed vs concise)

Prompt Template:

You are given a Python-based problem and a piece of buggy code that attempts to solve it but fails on a specific test case. Your task is to:

1. **Simulate the buggy code's execution** using the provided failing test case.
2. **Identify the root cause** of the failure by walking through the problematic parts of the code.
3. Enclose your refined code between '[Fix]'
4. **Fix the bug** and present the corrected solution.

[Buggy Code]

```
def bubble_sort(arr):
    # (1) [arr = [39, 12, 18, 85, 72, 10, 2, 18]]
    n = len(arr)
    # (2) n=8
    for j in range(0, n):
        # (3) [j = 0]; (5) [j = 1]; ...; (14) [j = 0]
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
        # (4) [arr = [12, 39, 18, 85, 72, 10, 2, 18]; ...;
        # (15) [arr = [2, 10, 12, 18, 18, 39, 72, 85]]
    return arr
# Return [12, 18, 39, 72, 10, 2, 18, 85]
```

[TestCase]

```
input=[39,12,18,85,72,10,2,18]
expect=[2, 10, 12, 18, 18, 39, 72, 85]

[ERROR] Traceback (most recent call last):
  File "code.py", line 9, in <module>
    assert bubble_sort(arr=[39,12,...,18])== [2,
    10, ..., 85]
AssertionError
```

Representation

Prompt
(a NExT concrete example)

Rationale and Buggy Code

```
def bubble_sort(arr):
    # (1) [arr = [39, 12, 18, 85, 72, 10, 2, 18]]
    for i in range(n):
        # (3) [i = 0]; (17) [i = 1]; ...; (55) [i = 7]
        for j in range(0, n-i-1):
            # (4) [j = 0]; (6) [j = 1]; ...; (54) [j = 0]
            arr[j], arr[j+1] = arr[j+1], arr[j]
            # (5) [arr = [12, 39, 18, 85, 72, 10, 2,
            18]];
            # (51) [arr = [2, 10, 12, 18, 18, 39, 72,
            85]]
    return arr
# __return__=[2, 10, 12, 18, 18, 39, 72, 85]
```

NExT

Rationale

[TEST1 RESULT]
Line 1: [arr = [39, 12, 18, 85, 72, 10, 2, 18]]
Line 2: [i = 0]; (17) [i = 1]; ...; (55) [i = 7]
Line 4: [arr = [12, 39, 18, 85, 72, 10, 2, 18]];
...
Line 54: [arr = [2, 10, 12, 18, 18, 39, 72, 85]]
Line 55: __return__=[2, 10, 12, 18, 18, 39, 72, 85]

Buggy Code

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
```

Concise

Rationale

Input: [39, ..., 72, 10, 2, 18]
Passes: n-1 = 4

Pass	Action	Array State
1 (i=0)	Swaps: 39↔10, 72↔10, 72↔2, 72↔18	[10, 2, 18, 39, 72]
2 (i=1)	Swaps: 10↔2, 18↔10	[2, 10, 18, 39, 72]
3 (i=2)	No swaps	[2, 10, 18, 39, 72]

Buggy Code

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
```

Code Executor

Rationale

Function Behavior:
1. **Outer Loop**: Runs multiple passes through the array. Each pass moves the largest unsorted element to its correct position at the end.
2. **Inner Loop**: Compares and swaps adjacent elements if needed.
3. **Output**: After all iterations, the array is sorted in ascending order.
How to fix:
1. **Type**: 'arr[j+1]' should be 'arr[j+1]', as 'l' is undefined.

Buggy Code

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
```

SemCoder

Evaluation Results

SFT

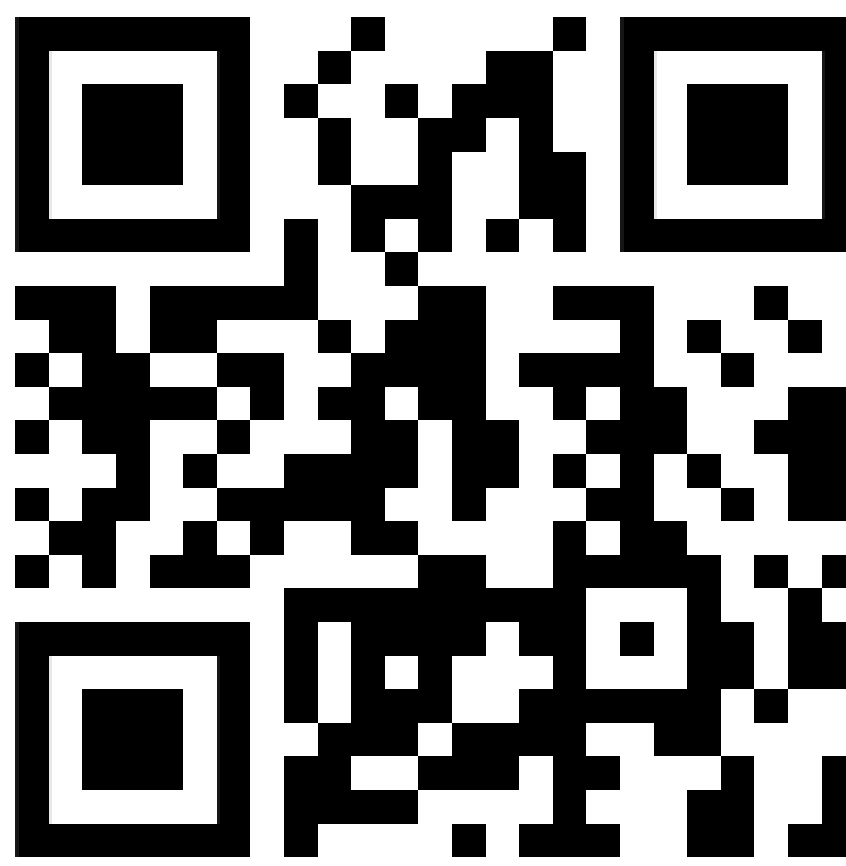
BaseModel	TrainCorpus	Finetune		Code Repair MBPP-R	Code Synthesis			Code Reasoning	
		downstream	trace		MBPP	BigcodeBench	LiveCodeBench	CRUXEval-I	CRUXEval-O
DeepSeek-Coder	-	✗	✗	17.7	71.9	41.5	40.8	40.0	40.4
	only NL2Code	✗	✗	25.4	72.9	43.7	12.6	60.1	55.4
	w/o trace	✓	✗	39.2	75.9	45.4	35.7	61.9	56.6
	Concise	✓	✓	39.2	74.4	44.3	29.4	61.6	55.0
	CodeExecutor	✓	✓	38.4	77.2	44.6	31.5	60.4	56.1
	NeXT	✓	✓	37.6	76.7	44.0	36.1	61.3	54.2
	SemCoder(GPT4o)	✓	✓	37.0	75.7	45.4	31.5	62.0	58.1
	SemCoder	✓	✓	40.5	76.4	45.7	29.0	59.5	55.4
LLaMA	-	✗	✗	20.1	58.6	31.4	27.3	42.6	36.2
	only NL2Code	✗	✗	24.9	73.7	44.1	18.1	60.1	55.9
	w/o trace	✓	✗	29.1	59.1	31.6	8.4	58.8	54.0
	Concise	✓	✓	27.0	59.4	30.4	14.7	55.8	57.6
	CodeExecutor	✓	✓	24.9	59.4	32.6	9.7	57.0	55.2
	NeXT	✓	✓	29.1	61.4	30.6	16.0	56.9	52.8
	SemCoder(GPT4o)	✓	✓	22.2	59.4	31.4	10.9	58.6	58.0
	SemCoder	✓	✓	29.4	61.9	33.4	14.7	59.9	55.4
Gemma2	-	✗	✗	20.9	63.7	29.8	32.8	49.2	41.5
	only NL2Code	✓	✗	19.8	61.4	26.8	12.6	57.9	55.6
	w/o trace	✓	✗	24.9	58.4	25.1	6.7	57.8	57.5
	Concise	✓	✓	22.8	60.2	28.8	8.0	57.6	57.2
	CodeExecutor	✓	✓	22.8	59.4	27.3	8.8	58.9	58.2
	NeXT	✓	✓	26.2	58.1	26.9	8.8	59.5	55.8
	SemCoder(GPT4o)	✓	✓	24.1	62.9	29.5	8.4	58.9	56.5
	SemCoder	✓	✓	26.2	62.2	27.6	13.0	58.9	56.8

Test-scaling

	Greedy	COT	Sequential Scaling				
			w/o trace	CodeExecutor	Concise	NExT	SemCoder
GPT-4o-mini	73.08	73.08	98.46	98.46	99.23	99.23	99.23
deepseek-chat(V3)	84.62	100.00	100.00	100.00	100.00	100.00	100.00
Reasoning Compatible Model							
AIDC-AI/Marco-o1	53.85	50.00	76.92	69.23	76.92	73.08	73.08
microsoft/phi-4	53.85	73.08	100.00	96.15	100.00	91.54	100.00
Instruction of Foundation Model							
Llama-3.1-8B-Inst	34.62	34.62	67.69	66.92	74.62	74.62	65.38
deepseek-coder-6.7b-Inst	42.31	46.15	68.46	61.54	69.23	76.15	69.23
Qwen2.5-Coder-7b-Inst	61.54	34.62	83.85	87.69	80.77	90.77	86.92

- Test-scaling consistently improves performance (65/70 cases)

- No single trace representation consistently wins, model-dependence.
- The SemCoder shows limited improvements (0.3-1.4 points)



Website